# Kegiatan Belajar 2.2: Perulangan, Fungsi, Array, dan Objek

## Hari 6 · 8 JP · UK 1.5 (utama), 1.3, dan 1.4

**Tujuan KB 2.2.** Setelah mengikuti KB ini, Anda mampu menggunakan perulangan, membuat fungsi, mengolah data dengan array dan objek, serta menulis kode yang rapi dengan penamaan dan komentar yang baik.

| Kegiatan | JP |
|---|---|
| Perulangan: `for`, `while`, `for...of` | 1 |
| Fungsi: parameter, nilai kembalian, prosedur | 2 |
| Array: membuat, mengakses, metode, dua dimensi, pengurutan | 2 |
| Objek dan array objek; `filter`, `find`, `map` | 2 |
| Penamaan, komentar, dan best practices; latihan dan cek kemampuan | 1 |
| **Jumlah** | **8** |

### Materi 1: Perulangan (*for, while, for…of*)

Perulangan menjalankan blok kode **berkali-kali**.

```
// for: jumlah pengulangan sudah diketahui
for (let i = 1; i <= 5; i++) {

console.log("Bungkus ke-" + i);
}
```

Bagian `for` terdiri atas tiga hal: **awal** (`let i = 1`), **syarat** (`i <= 5`, selama benar, ulangi), dan **langkah** (`i++`, tambah 1 tiap putaran).

```
// while: ulangi selama syarat benar
let sisa = 3;
while (sisa > 0) {
  console.log("Stok tersisa: " + sisa);
  sisa--;
}
```

**Awas perulangan tak berujung.** Jika `sisa--` terlupa, syarat `sisa > 0` selalu benar dan halaman akan macet. Pastikan setiap perulangan punya jalan keluar.

Perintah tambahan: `break` menghentikan perulangan, `continue` melompat ke putaran berikutnya.

### Materi 2: Fungsi (*function*)

**Fungsi** adalah blok kode bernama yang dapat dipakai berulang. Manfaatnya: kode lebih rapi, tidak berulang-ulang, dan mudah diuji.

```
// Fungsi dengan parameter dan nilai kembalian
function hitungTotal(harga, jumlah) {
  return harga * jumlah;
}

// Prosedur: fungsi yang hanya melakukan sesuatu, tanpa return
function tampilkanSapaan(nama) {
  console.log("Halo, " + nama + "!");
}

const total = hitungTotal(85000, 2);   // total bernilai 170000
tampilkanSapaan("Rahma");              // mencetak: Halo, Rahma!
```

| Istilah | Arti |
|---|---|
| Parameter | Nama masukan pada definisi fungsi (`harga`, `jumlah`) |
| Argumen | Nilai yang diberikan saat fungsi dipanggil (`85000`, `2`) |
| `return` | Mengembalikan hasil ke pemanggil, sekaligus mengakhiri fungsi |
| Prosedur | Fungsi tanpa `return`; hanya menjalankan aksi |

**Cakupan (scope).** Variabel yang dibuat di dalam fungsi hanya dikenal di dalam fungsi itu. Ini mencegah bentrok antarbagian program.

Fungsi juga dapat ditulis tanpa nama dan diberikan langsung sebagai argumen (dipakai pada `forEach`, `map`, dan `filter` nanti). Bentuk lain yang lebih singkat, disebut *arrow function*, `(p) => p.harga`, bekerja sama dengan `function (p) { return p.harga; }`. Pelatihan ini memakai bentuk `function` agar lebih jelas.

## Materi 3: Larik (*Array*)

**Array** menyimpan banyak/sekumpulan nilai dalam satu variabel, berurutan dengan **indeks yang dimulai dari 0**.

```
const kopi = ["Toraja", "Gayo", "Kintamani"];

console.log(kopi[0]);       // Toraja (indeks pertama = 0)
console.log(kopi.length);   // 3 (jumlah elemen)
kopi.push("Bajawa");        // menambah di akhir
console.log(kopi.includes("Gayo")); // true
```

Menelusuri array:

```
for (const jenis of kopi) {
  console.log(jenis);
}

kopi.forEach(function (jenis, indeks) {
  console.log((indeks + 1) + ". " + jenis);
});
```

**Array dua dimensi** adalah array yang elemennya juga array (seperti tabel baris dan kolom):

```
const stok = [
  ["Toraja", 20],
  ["Gayo", 15],
];
console.log(stok[1][0]);   // Gayo (baris ke-1, kolom ke-0)
console.log(stok[0][1]);   // 20
```

**Pengurutan.** Metode `sort` tanpa argumen mengurutkan sebagai teks. Untuk angka, berikan fungsi pembanding:

```
const harga = [85000, 60000, 90000, 80000];
harga.sort(function (a, b) {
  return a - b;              // naik (kecil ke besar)
});
console.log(harga);          // [60000, 80000, 85000, 90000]
```

**Catatan:** `sort` **mengubah array aslinya**. Untuk mengurutkan tanpa merusak data asli, salin dulu dengan `slice()`: `const urut = data.slice().sort(...)`.

## Materi 4: Objek dan Array Objek

**Objek** menyimpan data bernama (pasangan *properti* dan nilai) tentang satu benda.

```
const produk = {
  nama: "Kopi Toraja",
  asal: "Sulawesi Selatan",
  kategori: "arabika",
  harga: 85000,
};

console.log(produk.nama);    // Kopi Toraja
produk.harga = 90000;        // mengubah nilai properti
```

Data sungguhan biasanya berupa **array yang berisi objek**, seperti daftar produk di proyek kita. Tiga metode array yang sangat sering dipakai:

| Metode | Fungsi | Hasil |
|---|---|---|
| `filter` | Menyaring elemen yang memenuhi syarat | **Array baru** (bisa kosong) |
| `find` | Mencari **satu elemen pertama** yang memenuhi syarat | Satu elemen, atau `undefined` |
| `map` | Mengubah setiap elemen menjadi bentuk lain | Array baru dengan panjang sama |

```
const arabika = PRODUCTS.filter(function (p) {
  return p.kategori === "arabika";
});

const gayo = PRODUCTS.find(function (p) {
  return p.slug === "gayo";
});

const namaSaja = PRODUCTS.map(function (p) {
  return p.nama;
});
```

## Materi 5: Penamaan, Komentar, dan Best Practices

| Area | Praktik |
|---|---|
| Deklarasi | `const` secara bawaan; `let` hanya jika nilainya berubah; hindari `var` |
| Perbandingan | Gunakan `===` dan `!==` |
| Penamaan | camelCase yang bermakna: `hargaPerBungkus`, `hitungTotal`. Hindari `x`, `a1`, `data2` |
| Fungsi | Satu fungsi untuk satu tugas; nama fungsi diawali kata kerja (`hitung...`, `tampilkan...`) |
| Angka ajaib | Beri nama pada angka penting: `const TARIF_DISKON = 0.1;` |
| Komentar | `//` untuk satu baris, `/* ... */` untuk beberapa baris. Jelaskan **mengapa**, bukan hal yang sudah jelas |
| Format | Indentasi 2 spasi, titik koma konsisten, satu pernyataan per baris |

## Praktik 1: Membuat `data.js`

Buat folder `js` di proyek Kopi Nusantara, lalu buat berkas `js/data.js`. Berkas ini menyimpan **semua data** terpisah dari logika, dan akan dipakai lagi di KB 2.4.

```
// data.js: semua data ada di sini, terpisah dari logika program.
// Ingin menambah menu atau produk? Cukup ubah berkas ini.

const SITE = {
  nama: "Kopi Nusantara",
  email: "halo@kopinusantara.test",
};

const MENU = [
  { id: "beranda", label: "Beranda" },
  { id: "produk", label: "Produk" },
  { id: "kontak", label: "Kontak" },
];

const PRODUCTS = [
  { slug: "toraja", nama: "Kopi Toraja", asal: "Sulawesi Selatan",
    kategori: "arabika", harga: 85000,
    deskripsi: "Body tebal, aroma rempah dan cokelat gelap." },
  { slug: "gayo", nama: "Kopi Gayo", asal: "Aceh",
    kategori: "arabika", harga: 80000,
    deskripsi: "Rasa bersih dengan sentuhan karamel." },
  { slug: "kintamani", nama: "Kopi Kintamani", asal: "Bali",
    kategori: "arabika", harga: 90000,
    deskripsi: "Keasaman cerah seperti jeruk, ringan dan segar." },
  { slug: "bajawa", nama: "Kopi Bajawa", asal: "Flores",
    kategori: "arabika", harga: 88000,
    deskripsi: "Aroma floral dengan aftertaste manis." },
  { slug: "lampung", nama: "Robusta Lampung", asal: "Lampung",
    kategori: "robusta", harga: 60000,
    deskripsi: "Pahit tegas dan kental, cocok untuk kopi susu." },
  { slug: "temanggung", nama: "Robusta Temanggung", asal: "Jawa Tengah",
    kategori: "robusta", harga: 65000,
    deskripsi: "Rasa earthy dengan aroma kacang panggang." },
];
```

### Praktik 2: Program Keranjang Belanja

Buat `latihan/keranjang.html` (kerangka seperti Praktik 1, dengan `src="keranjang.js"`) dan `latihan/keranjang.js`:

```
// Program keranjang belanja: memakai array objek, perulangan, dan fungsi

const keranjang = [
  { nama: "Kopi Toraja", harga: 85000, jumlah: 2 },
  { nama: "Kopi Gayo", harga: 80000, jumlah: 1 },
  { nama: "Robusta Lampung", harga: 60000, jumlah: 3 },
];

function hitungSubtotal(item) {
  return item.harga * item.jumlah;
}

function hitungTotal(daftar) {
  let total = 0;
  for (const item of daftar) {
    total = total + hitungSubtotal(item);
  }
  return total;
}

function formatRupiah(angka) {
  return "Rp " + angka.toLocaleString("id-ID");
}

// Tampilkan rincian
keranjang.forEach(function (item) {
  console.log(item.nama + ": " + formatRupiah(hitungSubtotal(item)));
});
console.log("Total: " + formatRupiah(hitungTotal(keranjang)));
```

Hasil yang benar di Console: Kopi Toraja Rp 170.000, Kopi Gayo Rp 80.000, Robusta Lampung Rp 180.000, dan Total Rp 430.000. Jika berbeda, telusuri dengan `console.log` di setiap fungsi.